

Phase 2 — RAG sur la documentation Python

Cinq concepts essentiels du *Retrieval-Augmented Generation*

Cours PyTutor2

Résumé

Cette phase 2 construit, brique par brique, un système de *Retrieval-Augmented Generation* (RAG) qui répond à des questions en s'appuyant sur la documentation Python officielle. On y aborde : l'indexation (Web → embeddings → Chroma persistante), le **Retriever** LCEL, la chaîne RAG canonique en LCEL, la récupération des sources avec **RunnableParallel**, et la comparaison *similarity vs MMR*. Chaque exercice suit la même structure que la Phase 1 (énoncé + objectifs en boîte, solution détaillée, vérification LangSmith, récapitulatif).

Table des matières

1	Indexation : du Web vers la base vectorielle	2
1.1	Solution	2
1.1.1	Architecture	2
1.1.2	Le code complet	3
1.1.3	Explications pas à pas	4
1.2	Vérification dans LangSmith	5
2	Retriever : recherche par similarité	6
2.1	Solution	6
2.1.1	Architecture	6
2.1.2	Le code complet	6
2.1.3	Explications pas à pas	7
2.2	Vérification dans LangSmith	8
3	Chaîne RAG simple en LCEL	8
3.1	Solution	9
3.1.1	Architecture	9
3.1.2	Le code complet	9
3.1.3	Explications pas à pas	10
3.2	Vérification dans LangSmith	11
4	RAG avec sources : récupérer réponse <i>et</i> contexte	12
4.1	Solution	13
4.1.1	Architecture	13
4.1.2	Le code complet	13
4.1.3	Explications pas à pas	14
4.2	Vérification dans LangSmith	15
5	Recherche <i>similarity vs MMR</i>	16
5.1	Solution	16
5.1.1	Architecture	16
5.1.2	Le code complet	16
5.1.3	Explications pas à pas	17
5.2	Vérification dans LangSmith	18

1 Indexation : du Web vers la base vectorielle

Exercice 1. Construire un index Chroma persistant

Aspire 7 pages de la documentation officielle Python (docs.python.org/3/tutorial/), découpe-les en *chunks*, encode-les en vecteurs avec un modèle d'embeddings local, et persiste l'index sur disque dans un dossier `chroma_db/`.

Tu écriras une fonction `build_or_load_vectorstore(force_rebuild=False)` qui : (a) **re-charge** l'index depuis le disque si `chroma_db/` existe ; (b) **reconstruit** l'index si l'argument `force_rebuild` est vrai ou si le dossier n'existe pas.

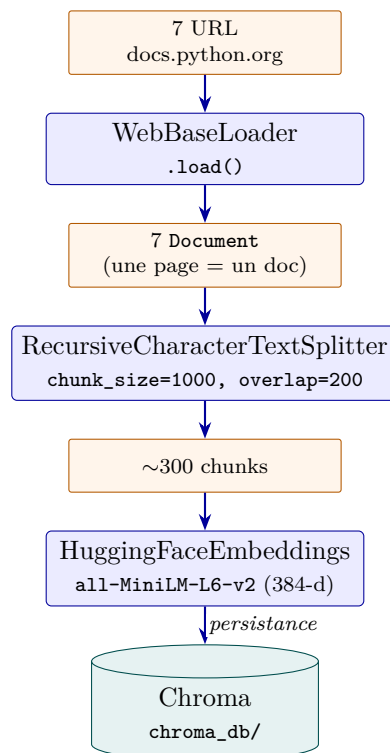
Contrainte : l'indexation initiale prend ~30 s (téléchargement des embeddings + scraping des URL) ; les lancements suivants doivent prendre <1 s grâce à la persistance.

Objectifs pédagogiques

- Utiliser `WebBaseLoader` pour aspirer une liste d'URL et obtenir des `Document` LangChain (texte + metadata).
- Choisir un `chunk_size` et un `chunk_overlap` cohérents pour de la doc technique, avec `RecursiveCharacterTextSplitter`.
- Encoder du texte en vecteurs avec `HuggingFaceEmbeddings` (modèle local, gratuit).
- Persister un `Chroma` sur disque et savoir le recharger sans réindexer.

1.1 Solution

1.1.1 Architecture



Un pipeline en quatre étapes : **scraping**, **découpage**, **encodage**, **stockage**. La séparation entre indexation (lente, faite une fois) et interrogation (rapide, à chaque question) est essentielle pour un RAG utilisable au quotidien.

1.1.2 Le code complet

```
1 from pathlib import Path
2
3 from langchain_community.document_loaders import WebBaseLoader
4 from langchain_text_splitters import RecursiveCharacterTextSplitter
5 from langchain_huggingface import HuggingFaceEmbeddings
6 from langchain_chroma import Chroma
7
8
9 PYTHON_DOCS_URLS = [
10     "https://docs.python.org/3/tutorial/controlflow.html",
11     "https://docs.python.org/3/tutorial/datastructures.html",
12     "https://docs.python.org/3/tutorial/classes.html",
13     "https://docs.python.org/3/tutorial/errors.html",
14     "https://docs.python.org/3/tutorial/modules.html",
15     "https://docs.python.org/3/howto/functional.html",
16     "https://docs.python.org/3/glossary.html",
17 ]
18
19 PERSIST_DIR = "./chroma_db"
20 EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
21 COLLECTION_NAME = "python_docs"
22
23
24 def get_embeddings() -> HuggingFaceEmbeddings:
25     """Singleton de fait : un seul modèle d'embeddings réutilisé partout."""
26     return HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL)
27
28
29 def build_or_load_vectorstore(force_rebuild: bool = False) -> Chroma:
30     """Recharge l'index si chroma_db/ existe, sinon le construit."""
31     embeddings = get_embeddings()
32     persist_path = Path(PERSIST_DIR)
33
34     if persist_path.exists() and not force_rebuild:
35         print(f"V Rechargement de l'index depuis {PERSIST_DIR}")
36         return Chroma(
37             collection_name=COLLECTION_NAME,
38             persist_directory=PERSIST_DIR,
39             embedding_function=embeddings,
40         )
41
42     # --- 1. Scraping : URL → Document ---
43     loader = WebBaseLoader(PYTHON_DOCS_URLS)
44     loader.requests_kwargs = {"headers": {"User-Agent": "PyTutor-Edu/0.1"}}
45     raw_docs = loader.load()
46
47     # --- 2. Découpage : Document → chunks ---
48     splitter = RecursiveCharacterTextSplitter(
49         chunk_size=1000,
50         chunk_overlap=200,
51         separators=["\n\n", "\n", ". ", " ", ""],
52     )
53     chunks = splitter.split_documents(raw_docs)
54
55     # --- 3. Encodage + 4. Persistance (en une étape) ---
56     vectorstore = Chroma.from_documents(
57         documents=chunks,
```

```

58     embedding=embeddings,
59     collection_name=COLLECTION_NAME,
60     persist_directory=PERSIST_DIR,
61 )
62 return vectorstore

```

1.1.3 Explications pas à pas

Étape 1 — WebBaseLoader : du Web au Document

WebBaseLoader(urls) accepte une URL ou une liste d'URL. Chaque page devient un `langchain_core.documents`

```

1 Document(
2     page_content="Texte complet de la page...",
3     metadata={"source": "https://docs.python.org/3/tutorial/classes.html",
4               "title": "9. Classes"},
5 )

```

Le champ `metadata["source"]` est précieux : il permettra plus tard de **citer les sources** et de filtrer.

User-Agent personnalisé : `loader.requests_kwargs = {"headers": {"User-Agent": "PyTutor-Edu/0.1"}}.` Certains sites bloquent les bots anonymes ; mieux vaut s'identifier explicitement.

Étape 2 — RecursiveCharacterTextSplitter : le découpage

Un Document de 50 000 caractères est inutilisable directement : trop gros pour le contexte du LLM, et la recherche par similarité perd en précision sur des blocs trop larges. On découpe en *chunks* :

```

1 RecursiveCharacterTextSplitter(
2     chunk_size=1000,                                # en CARACTÈRES
3     chunk_overlap=200,                              # 20% de chevauchement
4     separators=["\n\n", "\n", ". ", " ", ""],        # ordre prioritaire
5 )

```

Trois paramètres clés :

- **chunk_size=1000** : en caractères, *pas en tokens*. 1000 caractères $\approx$ 200–250 tokens — bon compromis pour de la doc technique.
- **chunk_overlap=200** : 20% de chevauchement entre chunks consécutifs. Évite de couper une idée en deux.
- **separators** : l'ordre compte. Le splitter essaie d'abord `"\n\n"` (paragraphe), puis `"\n"` (ligne), puis `". "` (phrase), et en dernier recours au milieu d'un mot.

La taille du chunk est l'hyperparamètre n°1 de qualité d'un RAG. Trop petit : contexte fragmenté, le modèle manque le sens global. Trop grand : le retriever distingue mal les passages pertinents des passages tangentiels. 800-1200 caractères est le sweet spot pour de la doc technique.

Étape 3 — HuggingFaceEmbeddings : encoder en vecteurs

L'embedding transforme une chaîne de caractères en vecteur dense de dimension fixe (~ 384 ici). Deux passages sémantiquement proches donnent des vecteurs proches en distance cosinus.

```
1 HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
```

Le modèle all-MiniLM-L6-v2 :

- **384 dimensions** (compact, rapide)
- **Multilingue** (utile pour des questions en français sur de la doc anglaise)
- **Local** (pas d'appel API, pas de coût par requête)
- **~80 MB** (téléchargé une fois, mis en cache)

Étape 4 — Chroma : stocker et rechercher

Chroma est une base de données vectorielle qui :

- Indexe les vecteurs pour la recherche par similarité.
- Stocke `page_content` et `metadata` à côté.
- Persiste dans un dossier sur disque (`persist_directory`).

`Chroma.from_documents(...)` fait tout en une étape : calcule les embeddings, indexe, persiste. Pour recharger plus tard :

```
1 Chroma(  
2     collection_name="python_docs",  
3     persist_directory="./chroma_db",  
4     embedding_function=embeddings,  # même modèle qu'à l'indexation !  
5 )
```

Étape 5 — Le pattern *lazy rebuild*

L'indexation est lente. On la fait une fois et on recharge ensuite :

```
1 if persist_path.exists() and not force_rebuild:  
2     return Chroma(...)          # rechargement <1s  
3 # sinon, indexation complète
```

C'est le pattern à connaître pour tous les artefacts coûteux à construire : embeddings, modèles fine-tunés, caches de prompts, etc.

1.2 Vérification dans LangSmith

L'indexation pure n'implique **pas** d'appel LLM, donc rien à voir dans LangSmith ici. Tu peux vérifier le contenu de l'index directement :

```
1 vs = build_or_load_vectorstore()  
2 print(vs._collection.count())      # nombre de chunks indexés  
3 docs = vs.similarity_search("lambda", k=3)  
4 for d in docs:  
5     print(d.metadata["source"], d.page_content[:100])
```

*LangSmith ne trace que les **Runnable** (chaînes, appels LLM, retrievers). L'indexation Chroma se passe en dehors de ce périmètre — c'est pourquoi on l'isole soigneusement du reste du code.*

Récapitulatif

Ce qu'il faut retenir :

1. Le pipeline d'indexation est universel : `Loader` → `Splitter` → `Embeddings` → `VectorStore`.
2. `chunk_size` est le **premier** hyperparamètre à régler. 800–1200 caractères pour de la doc technique.
3. `chunk_overlap` $\approx 20\%$ du `chunk_size` évite de couper une idée en deux.
4. `HuggingFaceEmbeddings` est local, gratuit, multilingue. L'alternative `OpenAIEmbeddings` est plus performante mais payante.
5. `Chroma` persiste sur disque. Pattern lazy rebuild : recharger si l'index existe, sinon reconstruire.

2 Retriever : recherche par similarité

Exercice 2. Transformer un vectorstore en Runnable LCEL

À partir d'un Chroma indexé, construis un Retriever qui prend une chaîne de caractères en entrée et renvoie les `k` documents les plus pertinents par similarité cosinus.

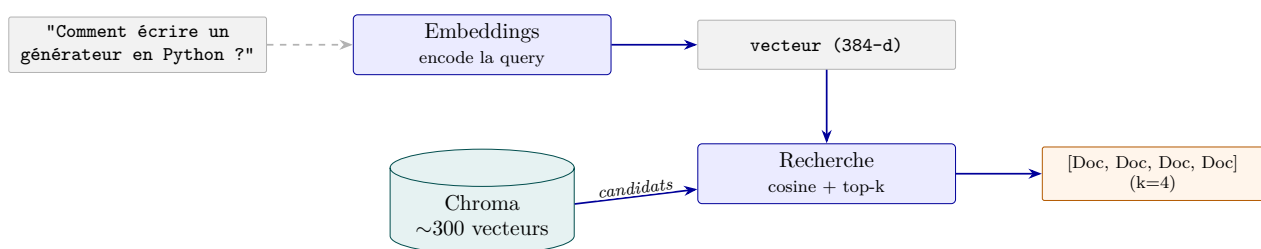
Contrainte : le Retriever doit être un Runnable LCEL, c'est-à-dire qu'il doit pouvoir être composé avec `|`, batché, streamé, et tracé dans LangSmith comme n'importe quel autre maillon.

Objectifs pédagogiques

- Comprendre comment `vectorstore.as_retriever()` produit un Runnable à partir d'un index Chroma.
- Paramétrer la recherche : `k` (nombre de docs retournés), `search_type` ("similarity" par défaut).
- Savoir lire ce qui se passe sous le capot : *embedding de la query* → *produit scalaire* → *tri* → *top-k*.

2.1 Solution

2.1.1 Architecture



Le retriever fait deux choses : (1) encoder la *query* avec le même modèle d'embeddings que les documents, (2) chercher les `k` vecteurs les plus proches par similarité cosinus.

2.1.2 Le code complet

```
1 # Le retriever est obtenu d'une seule ligne à partir du vectorstore.
2 vectorstore = build_or_load_vectorstore()
3
4 retriever = vectorstore.as_retriever(
5     search_type="similarity",      # défaut, on l'écrit pour la clarté
6     search_kwargs={"k": 4},        # nombre de docs renvoyés
7 )
```

```

8
9 # Le retriever est un Runnable : .invoke, .batch, .stream marchent
10 docs = retriever.invoke("Comment écrire un générateur en Python ?")
11
12 for i, d in enumerate(docs, 1):
13     print(f"[{i}] {d.metadata['source']}")
14     print(d.page_content[:200], "...")

```

2.1.3 Explications pas à pas

Étape 1 — .as_retriever() : du store au Runnable

Un Chroma (ou n'importe quel VectorStore LangChain) n'est pas directement un Runnable. La méthode .as_retriever() en crée un, qui peut alors être composé dans une chaîne LCEL avec l'opérateur |.

```

1 retriever = vectorstore.as_retriever(...)
2 # retriever est un Runnable
3 type(retriever).__mro__ # contient Runnable[str, list[Document]]

```

Le contrat de type : `str → list[Document]`.

Étape 2 — Ce qui se passe à l'.invoke()

1. La query (str) est encodée avec le **même HuggingFaceEmbeddings** qui a indexé les documents. C'est crucial : utiliser un autre modèle d'embeddings produirait un vecteur incomparable avec ceux de l'index.
2. Chroma calcule la similarité cosinus entre le vecteur query et **tous** les vecteurs indexés (en pratique avec un index approximatif pour aller vite).
3. Les k chunks de plus haute similarité sont renvoyés, triés par score décroissant.

Pourquoi cosinus et pas euclidienne ? La distance euclidienne dépend de la norme des vecteurs ; la similarité cosinus ne dépend que de l'angle. Pour des embeddings de phrases, c'est l'angle qui porte le sens.

Étape 3 — Le paramètre k

k est le nombre de chunks à renvoyer. C'est l'hyperparamètre n°2 de qualité du RAG (après chunk_size) :

- k=1 : trop pauvre, on rate souvent du contexte adjacent.
- k=4 à k=6 : standard pour de la doc technique.
- k=10+ : contexte saturé, le LLM peut perdre le fil. Coût en tokens augmenté.

Étape 4 — Le retriever dans une chaîne LCEL

Comme tout Runnable, le retriever se compose :

```

1 # Construire un mini-pipeline
2 def truncate(docs, n=3):
3     return docs[:n]
4
5 mini_chain = retriever | truncate
6 # Mini-chain reçoit une str et renvoie les 3 premiers docs

```

On verra dans l'exercice 3 comment l'enchaîner avec un prompt et un modèle pour faire un RAG complet.

2.2 Vérification dans LangSmith

Quand le retriever est utilisé dans une chaîne tracée, LangSmith affiche :

```
Retriever (VectorStoreRetriever)
Input : "Comment écrire un générateur en Python ?"
Output : [Document, Document, Document, Document]

[0] source: tutorial/classes.html
[1] source: howto/functional.html
[2] ...
```

*L'onglet Output montre la liste des **Document** renvoyés avec leur **metadata** (notamment **source**). C'est le **premier endroit où regarder** quand un RAG répond mal : souvent le retriever a ramené du contexte hors-sujet.*

Récapitulatif

Ce qu'il faut retenir :

1. `vectorstore.as_retriever(...)` transforme un store en **Runnable LCEL**, composable avec `|`.
2. Contrat du retriever : `str` $\rightarrow$ `list[Document]`.
3. Sous le capot : encode la query avec le même modèle d'embeddings, cherche les top-**k** par cosinus.
4. **k=4** à **k=6** est le standard pour de la doc technique.
5. Premier point de débogage d'un RAG cassé : regarder l'output du retriever dans LangSmith.

3 Chaîne RAG simple en LCEL

Exercice 3. Composer un RAG end-to-end en LCEL

Construis une chaîne **Runnable** qui prend une **question en texte brut** et renvoie une **réponse en texte brut**, en s'appuyant sur le retriever de l'exercice 2 pour récupérer du contexte pertinent dans la doc Python.

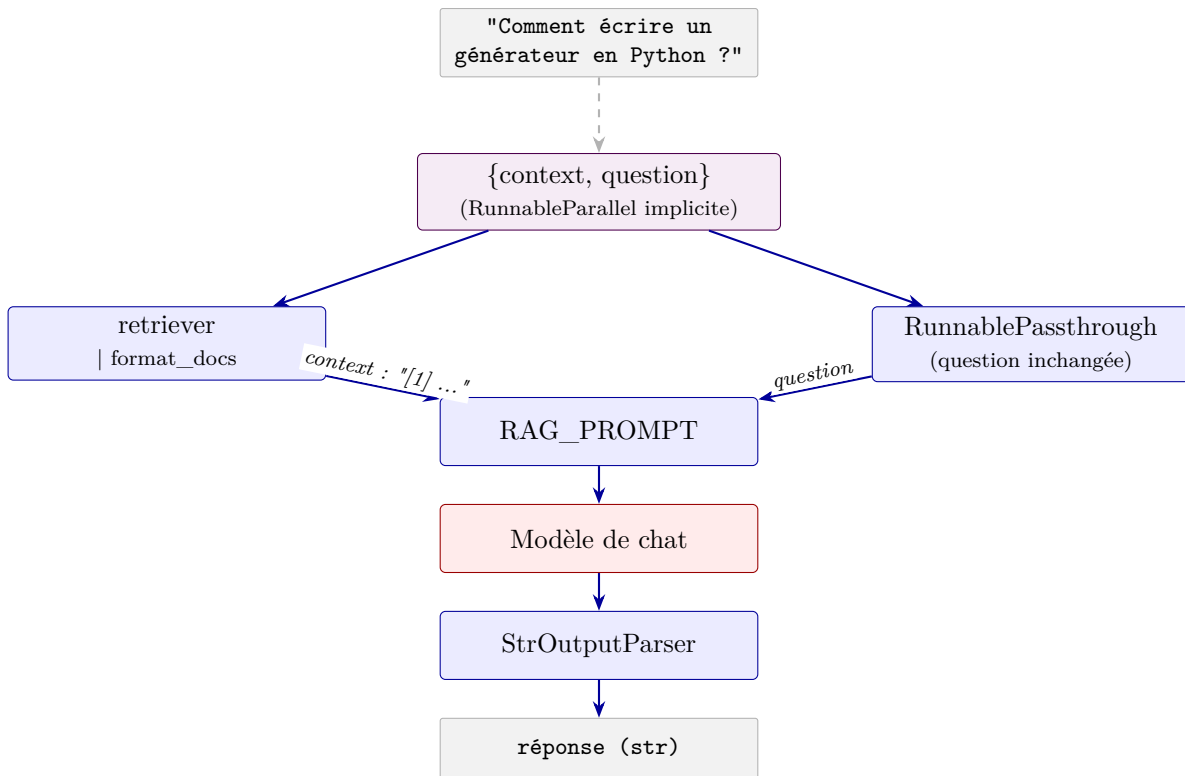
Contrainte : utiliser le pattern canonique LCEL avec un dict implicite qui contient deux branches `context` et `question`, puis composer `| prompt | model | StrOutputParser()`. Le modèle doit **citer ses sources** par numéro [N].

Objectifs pédagogiques

- Reconnaître le **pattern canonique RAG en LCEL** : `{context, question} | prompt | model | parser`.
- Utiliser `RunnablePassthrough()` pour faire passer la question à travers une branche du dict implicite.
- Composer `retriever | format_docs` pour transformer la liste de `Document` en string utilisable dans le prompt.
- Donner au modèle des instructions claires pour s'appuyer sur le contexte fourni et citer les sources.

3.1 Solution

3.1.1 Architecture



L'input (str) est dupliqué dans deux branches qui s'exécutent en parallèle : l'une va chercher le contexte, l'autre laisse passer la question. Les deux résultats sont assemblés dans un dict que le prompt sait formater.

3.1.2 Le code complet

```
1 from langchain_core.documents import Document
2 from langchain_core.prompts import ChatPromptTemplate
3 from langchain_core.output_parsers import StrOutputParser
4 from langchain_core.runnables import RunnablePassthrough
5 from config import get_model
6
7
8 RAG_PROMPT = ChatPromptTemplate.from_messages([
9     (
10         "system",
11         "Tu es un assistant pédagogique pour des élèves de Python. "
12         "Réponds à la question en t'appuyant UNIQUEMENT sur le contexte fourni.\n"
13         "Si la réponse n'est pas dans le contexte, dis-le explicitement plutôt "
14         "que d'inventer. Cite le ou les chunks utilisés par leur numéro entre "
15         "crochets, par exemple [1] ou [2,3].",
16     ),
17     (
18         "human",
19         "Contexte :\n{context}\n\n"
20         "Question : {question}\n\n"
21         "Réponse pédagogique :",
22     ),
23 ])
24
```

```

25
26 def format_docs(docs: list[Document]) -> str:
27     """Concatène les documents en un bloc de contexte numéroté."""
28     return "\n\n".join(
29         f"[{i + 1}] (source: {doc.metadata.get('source', '?')})\n{doc.page_content}"
30         for i, doc in enumerate(docs)
31     )
32
33
34 def build_rag_chain(retriever):
35     """Chaîne RAG simple : str -> str."""
36     return (
37         {
38             "context": retriever | format_docs,
39             "question": RunnablePassthrough(),
40         }
41         | RAG_PROMPT
42         | get_model()
43         | StrOutputParser()
44     )
45
46
47 # Usage
48 vectorstore = build_or_load_vectorstore()
49 retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
50 chain = build_rag_chain(retriever)
51
52 answer = chain.invoke("Comment écrire un générateur en Python ?")
53 print(answer)

```

3.1.3 Explications pas à pas

Étape 1 — Le dict implicite = RunnableParallel

```

1 {
2     "context": retriever | format_docs,
3     "question": RunnablePassthrough(),
4 }

```

Quand un dict apparaît dans une chaîne LCEL (au début ou via |), LCEL le convertit **implicitement** en `RunnableParallel`. Les deux branches reçoivent le même input (la question, str) et s'exécutent en parallèle :

- **branche context** : la string passe par `retriever` (qui renvoie une `list[Document]`), puis par `format_docs` (qui renvoie une str).
- **branche question** : la string traverse `RunnablePassthrough()` inchangée.

La sortie est un dict `{"context": "[1] ...", "question": "..."}` prêt à être consommé par le prompt.

*Pattern à mémoriser : { "key1": branch1, "key2": branch2 } | prompt | model
| parser. C'est la forme canonique du RAG en LCEL. Tu la reverras 1000 fois.*

Étape 2 — retriever | format_docs

`retriever` produit une `list[Document]`. Le prompt attend une chaîne de caractères pour la variable `{context}`. La composition `retriever | format_docs` chaîne les deux :

```

1 def format_docs(docs):
2     return "\n\n".join(
3         f"[{i+1}] (source: {d.metadata['source']})\n{d.page_content}"
4         for i, d in enumerate(docs)
5     )

```

Trois détails :

- Numérotation [1], [2], ... : indispensable pour que le modèle puisse citer les sources *par numéro*.
- Ajout de la `source` dans chaque bloc : traçabilité, et permet au modèle de citer l'URL.
- Séparateur "\n\n" : les LLM repèrent mieux les blocs distincts ainsi qu'avec un simple "\n".

Étape 3 — RunnablePassthrough

`RunnablePassthrough()` est l'identité de LCEL : il prend un input et le renvoie inchangé. Utile dans un `RunnableParallel` pour *conserver* une valeur d'entrée tout en faisant autre chose avec elle dans une autre branche.

```

1 RunnablePassthrough().invoke("hello")    # → "hello"

```

Étape 4 — Le prompt RAG

Le prompt système contient **trois consignes** essentielles :

1. « UNIQUEMENT sur le contexte fourni » — anti-hallucination.
2. « Si la réponse n'est pas dans le contexte, dis-le » — préserve l'honnêteté épistémique.
3. « Cite les chunks par numéro [N] » — traçabilité.

Les trois consignes du prompt RAG sont défensives. Les LLM modernes ont tendance à compléter avec leurs connaissances générales — en RAG, c'est exactement ce qu'on veut éviter (sinon, à quoi sert le retrieval ?). On verra dans l'exercice bonus 3 comment durcir encore le prompt.

Étape 5 — StrOutputParser : extraire le texte

Le modèle renvoie un `AIMessage`. Pour récupérer juste le texte de la réponse, on ajoute `| StrOutputParser()` en fin de chaîne. C'est le même parseur qu'on utilisait en streaming dans la Phase 1.

3.2 Vérification dans LangSmith

L'arbre d'exécution attendu :

```

RunnableSequence
  RunnableParallel<context, question>
    [context] RunnableSequence
      Retriever ← 1 query embedée
      format_docs (RunnableLambda)
    [question] RunnablePassthrough
      ChatPromptTemplate
      ChatAnthropic ← appel LLM
      StrOutputParser

```

Les trois points à regarder :

1. **Output du Retriever** : les chunks ramenés sont-ils pertinents ? Si non, problème côté indexation ou k .
2. **Input du ChatAnthropic** : le prompt rendu, avec le contexte complet. C'est la *vérité terrain* de ce que le modèle a vu.
3. **Output du ChatAnthropic** : la réponse cite-t-elle bien les [N] ? Si non, le modèle a peut-être ignoré le contexte (à durcir avec le prompt strict de l'exercice bonus 3).

*Si la réponse est incorrecte, **ne blâme pas le modèle d'abord** : regarde l'output du retriever. 80% des bugs RAG viennent d'un mauvais retrieval (`chunk_size` mal réglé, `query` mal formulée, k trop petit).*

Récapitulatif

Ce qu'il faut retenir :

1. *Pattern canonique RAG : {context, question} | prompt | model | parser.*
2. *Le **dict implicite** devient un `RunnableParallel` à deux branches : une qui va chercher (retriever / format), une qui passe la question.*
3. *`format_docs` numérote les chunks et expose les sources — condition pour que le modèle puisse citer.*
4. *Le prompt système doit explicitement : (1) restreindre au contexte, (2) autoriser le « je ne sais pas », (3) exiger les citations [N].*
5. *Pour debugger : lire d'abord l'output du retriever dans `LangSmith`, ensuite le prompt rendu.*

4 RAG avec sources : récupérer réponse *et* contexte

Exercice 4. Préserver les documents sources dans la sortie

Modifie la chaîne RAG de l'exercice 3 pour qu'elle renvoie un **dict** `{"answer": str, "context": list[Document], "question": str}` au lieu de juste la réponse en string. Cela permet d'afficher les sources à l'utilisateur et de *vérifier* sur quels chunks la réponse s'est appuyée.

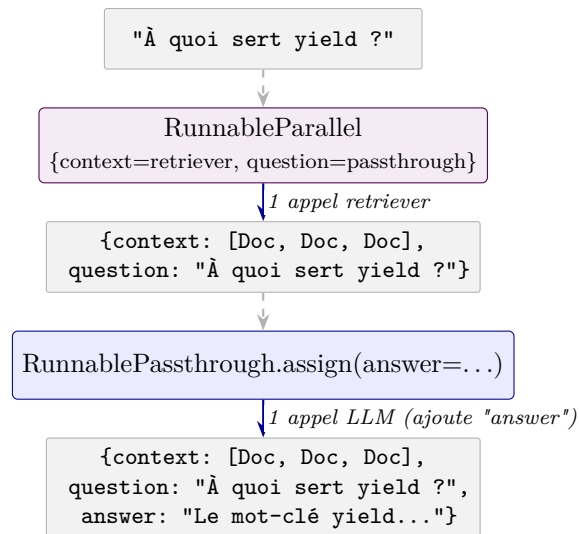
Contrainte : une seule invocation du retriever (pas deux). On utilise `RunnablePassthrough.assign` pour *enrichir* le dict au fil de la chaîne sans perdre les clés précédentes.

Objectifs pédagogiques

- Distinguer `RunnableParallel` (remplace l'input par un dict) de `RunnablePassthrough.assign` (enrichit le dict).
- Construire une chaîne en **deux phases** : récupération du contexte (1 appel retriever), puis génération de la réponse en gardant le contexte en mémoire.
- Comprendre le pattern *fan-out / accumulate* de LCEL.

4.1 Solution

4.1.1 Architecture



Deux étapes propres : d'abord on récupère le contexte une seule fois (branche `retrieve`), ensuite on ajoute la réponse au dict existant (`.assign`).

4.1.2 Le code complet

```
1 from langchain_core.runnables import RunnableParallel, RunnablePassthrough
2
3
4 def build_rag_chain_with_sources(retriever):
5     """Chaîne RAG qui renvoie un dict {answer, context, question}."""
6
7     # Étape 1 : récupère le contexte UNE FOIS et le passe en aval
8     retrieve = RunnableParallel(
9         context=retriever,
10        question=RunnablePassthrough(),
11    )
12
13    # Étape 2 : à partir de {context: [Doc...], question: str},
14    # construit la réponse en formatant les docs au passage.
15    answer_chain = (
16        RunnablePassthrough.assign(
17            context=lambda x: format_docs(x["context"]),
18        )
19        | RAG_PROMPT
20        | get_model()
21        | StrOutputParser()
22    )
23
24    # .assign(answer=...) ajoute la clé "answer" sans toucher aux autres.
25    return retrieve | RunnablePassthrough.assign(answer=answer_chain)
26
27
28 # Usage
29 chain = build_rag_chain_with_sources(retriever)
30 result = chain.invoke("À quoi sert le mot-clé yield exactement ?")
31
32 print("RÉPONSE :")
33 print(result["answer"])
```

```

34
35 print("\nSOURCES :")
36 for i, doc in enumerate(result["context"], 1):
37     print(f"    [{i}] {doc.metadata['source']}")
38     print(f"        {doc.page_content[:120]}...")

```

4.1.3 Explications pas à pas

Étape 1 — Récupérer une seule fois

```

1 retrieve = RunnableParallel(
2     context=retriever,
3     question=RunnablePassthrough(),
4 )

```

À partir d'une question (str), ce `RunnableParallel` produit :

- `context` : la `list[Document]` renvoyée par le retriever (**1 appel**).
- `question` : la string d'origine (passthrough).

Important : on ne formate *pas encore* les docs ici. On veut les garder bruts pour pouvoir les retourner à l'utilisateur en sortie.

Étape 2 — La sous-chaîne qui calcule la réponse

```

1 answer_chain = (
2     RunnablePassthrough.assign(
3         context=lambda x: format_docs(x["context"]),
4     )
5     | RAG_PROMPT | get_model() | StrOutputParser()
6 )

```

Cette sous-chaîne reçoit `{context: [Doc...], question: str}` et fait trois choses :

1. **Remplace** la clé `context` par sa version formatée en string. Le `RunnablePassthrough.assign(context=...)` applique la lambda à l'input courant et place le résultat dans `context`. La clé `question` est intacte.
2. Le prompt formate avec les variables `{context}` (str maintenant) et `{question}`.
3. Modèle + parser produisent la réponse en string.

`RunnablePassthrough.assign(key=fn)` remplace la clé `key` si elle existe déjà, ou la crée si elle n'existe pas. C'est l'équivalent d'un `UPDATE` en `SQL` sur un dict.

Étape 3 — L'accumulation finale

```

1 return retrieve | RunnablePassthrough.assign(answer=answer_chain)

```

Le `RunnablePassthrough.assign(answer=answer_chain)` prend le dict produit par `retrieve` (`{context: [Doc...], question: str}`) et y ajoute la clé `answer` (résultat de `answer_chain`).

La sortie finale :

```

1 {
2     "context": [Document, Document, Document, Document], # bruts, intacts

```

```

3     "question": "À quoi sert yield ?",                # str
4     "answer":   "Le mot-clé yield...",                # str
5 }

```

L'utilisateur peut afficher `result["answer"]` et énumérer `result["context"]` pour montrer les sources.

Étape 4 — Pourquoi pas deux retrievers ?

Une approche naïve appellerait deux fois le retriever : une fois pour afficher les sources, une fois inclus dans la chaîne RAG. C'est deux fois plus de calcul et l'index pourrait même retourner des docs différents (selon la non-déterminisme).

Le pattern `RunnableParallel` + `.assign` :

- **1 seul appel** retriever (économie).
- **Cohérence** : la réponse s'appuie *exactement* sur les sources affichées (pas de divergence possible).
- **Composable** : tout reste un `Runnable`.

4.2 Vérification dans LangSmith

L'arbre d'exécution est nettement plus riche :

```

RunnableSequence
  RunnableParallel<context, question>
    [context] Retriever
    [question] RunnablePassthrough
      RunnableAssign<answer>
        RunnableAssign<context=format_docs>
          ChatPromptTemplate
            ChatAnthropic ← appel LLM
          StrOutputParser

```

Dans l'onglet Output du run racine, tu vois directement le dict final avec ses 3 clés. C'est la matérialisation du pattern fan-out / accumulate qu'on cherche en LCEL pour produire des sorties riches sans dupliquer les calculs.

Récapitulatif

Ce qu'il faut retenir :

1. `RunnableParallel` crée un dict; `Passthrough.assign` enrichit un dict existant. Ne pas confondre.
2. Pattern fan-out / accumulate : une étape construit un dict, les étapes suivantes ajoutent des clés sans détruire les précédentes.
3. Une seule invocation du retriever — la réponse et les sources affichées sont cohérentes par construction.
4. Pour faire un RAG « professionnel » avec sources, c'est le pattern à connaître.

5 Recherche *similarity* vs *MMR*

Exercice 5. Comparer deux stratégies de retrieval

Compare deux `Retriever` sur la même question : l'un avec `search_type="similarity"` (par défaut), l'autre avec `search_type="mmr"` (*Maximal Marginal Relevance*). Observe que MMR produit des chunks plus *diversifiés* pour une question généraliste comme « *liste vs tuple* ».

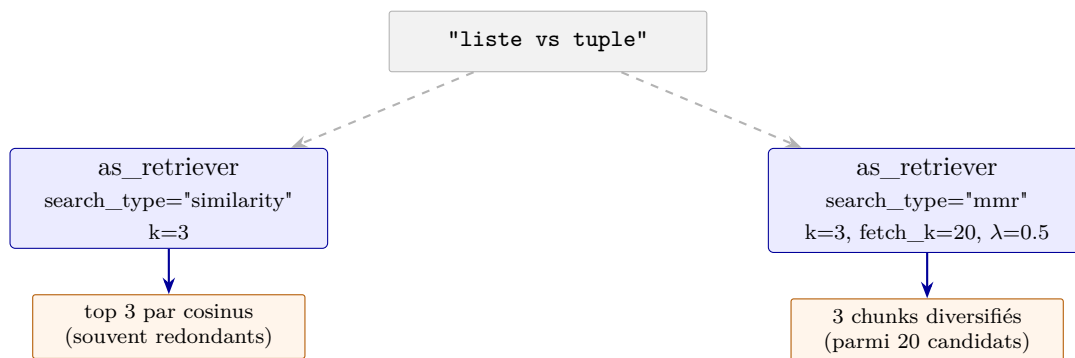
Contrainte : utiliser le même `Chroma`, juste changer `search_kwargs`. Pour MMR, expérimenter avec `fetch_k` (taille de la pool de candidats) et `lambda_mult` (équilibre similarité/diversité).

Objectifs pédagogiques

- Comprendre le problème de la *redondance sémantique* en retrieval pur par similarité.
- Connaître la *Maximal Marginal Relevance* (MMR) et ses deux paramètres : `fetch_k` et `lambda_mult`.
- Savoir choisir entre les deux stratégies selon la nature de la question (précise vs généraliste).

5.1 Solution

5.1.1 Architecture



5.1.2 Le code complet

```
1 vectorstore = build_or_load_vectorstore()
2 question = "Quelle est la différence entre une liste et un tuple ?"
3
4 # --- Retriever similarity (défaut) ---
5 sim_retriever = vectorstore.as_retriever(
6     search_type="similarity",
7     search_kwargs={"k": 3},
8 )
9
10 # --- Retriever MMR ---
11 mmr_retriever = vectorstore.as_retriever(
12     search_type="mmr",
13     search_kwargs={
14         "k": 3, # nombre final de docs renvoyés
15         "fetch_k": 20, # pool de candidats avant sélection MMR
16         "lambda_mult": 0.5, # 1.0 = pure similarité, 0.0 = pure diversité
17     },
18 )
19
20 # --- Comparaison ---
21 print("---- Top-3 similarity ----")
22 for i, doc in enumerate(sim_retriever.invoke(question), 1):
```



```

23     print(f" [{i}] {doc.page_content[:100]}...")
24
25 print("\n--- Top-3 MMR (diversifié) ---")
26 for i, doc in enumerate(mmr_retriever.invoke(question), 1):
27     print(f" [{i}] {doc.page_content[:100]}...")

```

5.1.3 Explications pas à pas

Étape 1 — Le problème de la similarité pure

La similarité cosinus classe les chunks par **proximité avec la query**, indépendamment les uns des autres. Conséquence : si la query matche fortement un certain passage, les **k** chunks les plus similaires sont souvent **très proches entre eux** :

Pour la query « liste vs tuple », les top-3 par similarité sont typiquement 3 paragraphes de la même section qui parlent de la même définition formelle. On rate les autres angles du sujet (mutabilité, performances, conventions d'usage).

Étape 2 — La *Maximal Marginal Relevance*

MMR sélectionne itérativement les chunks en équilibrant deux critères :

- **Pertinence** : similarité avec la query.
- **Diversité** : *dissimilarité* avec les chunks déjà sélectionnés.

Algorithme :

1. Récupérer les `fetch_k` candidats par similarité pure (ex : 20).
2. Sélectionner le candidat le plus proche de la query (top 1).
3. Pour chaque candidat restant, calculer :

$$\text{score} = \lambda \cdot \text{sim}(c, q) - (1 - \lambda) \cdot \max_{s \in S} \text{sim}(c, s)$$

où S est l'ensemble des candidats déjà sélectionnés.

4. Sélectionner le candidat de plus haut score. Répéter jusqu'à **k**.

Étape 3 — Les paramètres `fetch_k` et `lambda_mult`

Paramètre	Effet
<code>k</code>	Nombre final de docs renvoyés.
<code>fetch_k</code>	Taille de la pool de candidats sur laquelle MMR opère. Plus c'est grand, plus la diversité <i>potentielle</i> est large. Typique : <code>fetch_k = 5 × k</code> .
<code>lambda_mult</code>	Curseur similarité ↔ diversité : <code>1.0</code> = MMR redevient similarité pure, <code>0.0</code> = pure diversité (déconnectée de la query). Sweet spot : <code>0.5</code> à <code>0.7</code> .

Étape 4 — Quand utiliser MMR

similarity (défaut)	mmr
Question précise, monolithique	Question généraliste, multifacette
« Comment fonctionne <code>super()</code> ? »	« C'est quoi la POO en Python ? »
On veut le <i>meilleur</i> match exact	On veut un <i>tour d'horizon</i>
Plus rapide (pas de recalcul)	Plus lent ($O(k \times \text{fetch_k})$)

***Règle pratique** : si tes top-3 par similarité semblent redondants (3 passages disant la même chose), passe en MMR. Si les top-3 sont déjà variés, similarité suffit (et c'est plus rapide).*

5.2 Vérification dans LangSmith

Pour les deux retrievers, LangSmith affiche le même type de run (`VectorStoreRetriever`). La différence se voit dans le *contenu* des chunks renvoyés : tu compares visuellement les deux runs côte-à-côte dans l'UI.

LangSmith n'expose pas (encore) les scores MMR internes. Si tu veux les inspecter, utilise `vectorstore.max_marginal_relevance_search_with_score(...)` directement, en dehors de la chaîne.

Récapitulatif

Ce qu'il faut retenir :

- 1. La similarité pure peut être **redondante** : les top-**k** sont souvent des paragraphes proches du même passage.*
- 2. MMR équilibre **pertinence et diversité** : pioche dans une pool de `fetch_k` candidats les **k** les plus complémentaires.*
- 3. `lambda_mult` pilote le curseur : 0.5–0.7 est le sweet spot pour de la doc technique.*
- 4. Bascule en MMR pour les questions **généralistes ou ouvertes** ; reste en similarity pour les questions précises.*
- 5. Côté API : c'est juste un paramètre `search_type` à changer dans `.as_retriever(...)`.*